



µLab 1 • Week 2 / Day 2 • Driver's handout

Counting words in Spark on a larger dataset

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

STEP 1 • BRING THE CLUSTER BACK, AND RE-RUN MONDAY'S COUNT

Start Monday's cluster, put the new corpus where it can read it, and set the cluster to one worker. The corpus is for step 3; this step re-runs the count you already know, unchanged, so that steps 2 and 3 have something to be different from.

ON YOUR LAPTOP

```
docker start k8s
cp ~/Downloads/simplewiki.txt.bz2 lab/data/
```

`docker start k8s` Starts the stopped container that holds the cluster. If it answers `No such container`, use panel 7.
`cp ... lab/data/` `lab` is mounted into the cluster at `/lab`, so a file dropped here is visible to every pod.

Now open a **kubectl client**. You did this on 21.09 and the command has not changed: µLab 1, panel 1, **Start the cluster, then get a client**. Look it up on that sheet. Everything below runs in the shell it gives you.

IN THE KUBECTL CLIENT

```
kubectl scale deployment spark-worker --replicas=1
kubectl get pods
```

Wait for **one** `spark-worker`, `Running`. Then run Monday's count, exactly as Monday ran it:

```
kubectl exec deploy/spark-master -- \
/opt/spark/bin/spark-submit --master spark://spark-master:7077 \
/lab/wordcount_df.py /lab/data '/lab/data/20news/*.txt'
```

`kubectl exec deploy/spark-master --` Runs the command inside the master pod. Everything after `--` is that command.
`--master spark://spark-master:7077` Where to submit. `spark-master` is a cluster DNS name, `7077` the port the Service publishes.
`/lab/data` First argument: the folder holding `stopwords.txt`. It is not what gets counted.
`'/lab/data/20news/*.txt'` Second argument: what gets read. A pattern, not a folder.

input partitions _____ elapsed _____

The run prints fifteen words with their counts. Copy the **first three** and the **last three**, word and number both. Six lines is enough to recognise this answer again, and step 2 asks you to.

FIRST THREE

1	_____
2	_____
3	_____

LAST THREE

13	_____
14	_____
15	_____

The quotes around `'/lab/data/20news/*.txt'` are not decoration. From the shell you just typed that command in, run:

```
ls /lab/data
```

It almost certainly does not exist there: the client container was started without mounting `/lab`, so that path means nothing to the shell. The run works without the quotes anyway. **Say why the quotes are important even so.**

STEP 2 · STORE THE SAME TEXT AS ONE FILE, AND RUN IT AGAIN

What this is meant to show, before you run it. Step 1 read 18 846 files. Now glue them into a single compressed file and read that instead. Not one byte of text changes, so the answer cannot change. Everything about how Spark goes about it can.

ON YOUR LAPTOP

```
cd lab/data
find 20news -name '*.txt' -exec cat {} + | bzip2 > 20news.txt.bz2
ls -lh 20news.txt.bz2
cd ../../
```

`find ... -exec cat {} +` Prints every file, one after another, into one stream. The `+` hands `cat` many files per call instead of one, which is the difference between three seconds and a minute. Every message already ends in a newline, so nothing is glued to the end of the line before it.

`| bzip2 >` Compresses that stream into one file. `bzip2` is used rather than `gzip` for a reason you will meet in μ Lab 2.

It takes a few seconds and makes one file of about 11 MB. Then run the same count on it, from the client:

```
kubectll exec deploy/spark-master -- \
/opt/spark/bin/spark-submit --master spark://spark-master:7077 \
/lab/wordcount_df.py /lab/data /lab/data/20news.txt.bz2
```

`/lab/data/20news.txt.bz2` The only argument that differs from step 1: one file in place of a pattern matching 18 846 of them.

input partitions _____ elapsed _____

Now check the thing that matters. Compare the first three and last three lines with the six you copied in step 1.

Same six lines, or not? _____

STEP 3 · NOW RUN IT ON THE LARGER CORPUS

Same kind of file as step 2, but eight times as much text: one compressed file of 212 354 Wikipedia articles.

```
kubectll exec deploy/spark-master -- \
/opt/spark/bin/spark-submit --master spark://spark-master:7077 \
/lab/wordcount_df.py /lab/data /lab/data/simplewiki.txt.bz2
```

`/lab/data/simplewiki.txt.bz2` Again the only argument that differs. Three runs, one command, one changing word.

input partitions _____ elapsed _____

Step 2 held the text fixed and changed how it was stored. Step 3 held the storage fixed and used eight times as much text. One of those changed the partition count and one changed the clock. Write one sentence saying which did which.

YOUR THREE RUNS, SIDE BY SIDE

run	text	disk	partitions	run time
step 1 · 18 846 files	34.2 MB	87.9 MB	_____	_____
step 2 · the same text, one .bz2	34.2 MB	11.3 MB	_____	_____
step 3 · Wikipedia, one .bz2	272.8 MB	79.9 MB	_____	_____

text is how much text there is; **disk** is what it occupies, measured with `du`. Fill in the last two columns from your own three runs.

Solved means steps 1 to 3 are done and the table above is filled in. The referee ticks the bar below at that moment, not at the caucous.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucous. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

4 • THE SCRIPT YOU HAVE BEEN RUNNING

The comments have been taken out. Put them back: write on the right, beside each part, what that part of the script is doing.

```
"""Count the words in a text corpus, with the DataFrame API."""
import sys
import time
from pathlib import Path

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

DATA = Path(sys.argv[1]) if len(sys.argv) > 1 else Path("data")
INPUT = sys.argv[2] if len(sys.argv) > 2 else str(DATA / "20news" / "*.txt")
TOP_N = 15

stopwords = {
    line.strip().lower()
    for line in (DATA / "stopwords.txt").read_text().splitlines()
    if line.strip()
}

spark = SparkSession.builder.appName("wordcount-df").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

lines = spark.read.text(INPUT)

words = lines.select(
    F.explode(F.split(F.lower(F.col("value")), r"\W+")).alias("word")
)

words = words.filter(
    (F.length(F.col("word")) >= 3)
    & (~F.col("word").rlike(r"[0-9]+$"))
    & (~F.col("word").isin(list(stopwords)))
)

counts = words.groupBy("word").count().orderBy(F.desc("count"))

print("\n=== the plan, before a single byte has been read ===")
counts.explain()

print("\n=== now an action, and only now does anything run ===")
t0 = time.perf_counter()
rows = counts.take(TOP_N)
elapsed = time.perf_counter() - t0

for r in rows:
    print(f"{r['count']}>8} {r['word']}")
print(f"\ninput partitions : {lines.rdd.getNumPartitions()}")
print(f"elapsed          : {elapsed:.1f} s")

spark.stop()
```

WHAT IS THIS PART DOING?

5 • EXTENSION • CHANGE WHAT THE SCRIPT CALLS A WORD

The `words.filter(...)` block throws tokens away before counting: anything under three characters, anything that is all digits, and anything in `stopwords.txt`. Each of those is a decision somebody made, and each one changes the answer.

Make one change at a time, re-run, and record what moves:

- drop the length filter** Remove the `F.length(...) >= 3` line. A comment beside it **in the file** predicts what you will see: the listing above is trimmed of comments, the file is not. Was it right?
- raise TOP_N** `TOP_N`, near the top. Show fifty words instead of fifteen. Does the run take longer? Should it?
- add one stopword** `websites` is in the top ten of the Wikipedia corpus and nobody wrote it in a sentence: it comes from the **Other websites** heading. Add it and see what takes its place.

Then the question that matters: is editing the stopword list **cleaning the data** or **choosing the answer**? One sentence.

6 • EXTENSION • WHY 589, AND WHERE A SETTING LIVES

Step 1's 589 is neither one piece per file nor one in total. Spark groups small files, because opening a file costs time whatever its size, and it charges every file a flat fee to account for that. The fee is a setting with a default of 4 MiB, and you can change it for one run:

```
kubectll exec deploy/spark-master -- \
/opt/spark/bin/spark-submit --master spark://spark-master:7077 \
--conf spark.sql.files.openCostInBytes=1048576 \
/lab/wordcount_df.py /lab/data '/lab/data/20news/*.txt'
```

`--conf spark.sql.files.openCostInBytes` The fee, in bytes. 1048576 is 1 MiB, a quarter of the default. `--conf` sets it for this run only.

A cheaper fee means more files fit in one piece. Predict the direction, then read it off.

predicted: up or down _____ input partitions _____

The same setting can live in the script instead, where it is part of the program rather than part of how you happened to launch it. The line that builds the session takes it:

```
spark = (SparkSession.builder.appName("wordcount-df")
        .config("spark.sql.files.openCostInBytes", 1048576)
        .getOrCreate())
```

Copy the script to a new name, make that edit, and run it **without** `--conf`. Then say which of the two places you would use for something that must hold every time the script runs.

*Observed 2026-09-23: 148 pieces either way, down from 589. Edit under a **new** filename; a file rewritten in place is sometimes still the old one inside the pod.*

7 • ANNEX • IF THE CLUSTER IS GONE

`docker start k8s` answers No such container only if you removed the cluster rather than stopping it. Rebuilding it is not new work and it is not reprinted here: it is **µLab 3 of 21.09, panel 1, Start again from nothing**, which creates the container, starts a client with the manifest mounted, and applies it. Work through that panel, then come back to step 1 above.

Tell the Lab Master before you start: it takes several minutes, and somebody in the room has already done it today.